user only provides values for the first three countries, the final two will have their properties set to the default for the type—`null` for strings. So you only map those that have non-null values to the `Countries` collection in the `PageModel`. Then they are rendered along with their flag icon (if you entered valid ISO codes).

USING AN EXPLICIT INDEX

The approach that relies on an explicit index is more suited to forms designed for editing existing values, where a unique identifier, such as a database primary key of each item, is often used as the index value. In addition to incorporating an index value in the name of the form controls, this approach requires an additional hidden field for each item, named `<property>.Index`, which has a value set to the unique identifier. This is used by the model binder to group related controls together.

To see this working, you are going to simulate editing countries in bulk. First you need to modify the `Country` class to include an integer property named `Id`, which will be used to store the country's unique identifier. This is shown in bold in the next listing.

Listing 5.19 Adding a unique identifier property to the `Country` class

```
public class Country
{
    public int Id { get; set; }
    public string CountryName { get; set; }
    public string CountryCode { get; set; }
}
```

Next, add a new Razor page to the CountryManager folder named Edit. You are going to use an input model to represent the editable properties of countries, and then instantiate a collection of them in the `OnGet` handler.